

Ski Resort Data Analysis - Final Paper

Arnav, Sam, Quinn, and Santiago

2024-12-01

```
library(tidyverse)
library(tidymodels)
library(ggplot2)
library(dplyr)
library(knitr)
library(reshape2)
set.seed(123)
```

Reading and Cleaning Data

```
resort_df <- read_csv("resorts.csv")
```

Rows: 499 Columns: 25

-- Column specification -----

Delimiter: ","

chr (8): Resort, Country, Continent, Season, Child friendly, Snowparks, Nig...

dbl (17): ID, Latitude, Longitude, Price, Highest point, Lowest point, Begin...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
snow_df <- read_csv("snow.csv")
```

Rows: 820522 Columns: 4

-- Column specification -----

Delimiter: ","

dbl (3): Latitude, Longitude, Snow

```
date (1): Month
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
metadata <- read_csv("data_dictionary.csv")
```

```
Rows: 29 Columns: 3
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (3): Table, Field, Description
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
kable(head(resort_df |> select(ID, Resort, Latitude, Longitude, Country, Price, `Highest point
```

ID	Resort	Latitude	Longitude	Country	Price	Highest point	Beginner slopes
1	Hemsedal	60.92824	8.383487	Norway	46	1450	29
2	Geilosiden Geilo	60.53453	8.206372	Norway	44	1178	18
3	Golm	47.05781	9.828167	Austria	48	2110	13
4	Red Mountain Resort-Rossland	49.10552	- 117.846280	Canada	60	2075	20
5	Hafjell	61.23037	10.529014	Norway	45	1030	33
6	Voss	60.68371	6.407905	Norway	43	960	25

```
kable(head(snow_df))
```

Month	Latitude	Longitude	Snow
2022-12-01	63.125	68.875	95.28
2022-12-01	63.125	69.125	100.00
2022-12-01	63.125	69.375	100.00
2022-12-01	63.125	69.625	100.00
2022-12-01	63.125	69.875	100.00
2022-12-01	63.125	70.375	100.00

```
kable(head(metadata))
```

Table	Field	Description
Resorts	ID	Unique identifier for each resort
Resorts	Resort	Name of the ski & snowboard resort
Resorts	Latitude	Latitude for the resort's location
Resorts	Longitude	Longitude for the resort's location
Resorts	Country	Country in which the resort is located
Resorts	Continent	Continent in which the resort is located

Dataset was found on https://www.mavenanalytics.io/data-playground?order=date_added%2Cdesc&search=s
The csvs have been generated through webscraping from <https://ski-resort-stats.com/> and
the snow data come from <https://nsidc.org/data/mod10cm/versions/61>.

```
clean_non_alphanumeric <- function(data_col) {  
  # Convert non-ASCII characters to ASCII  
  data_col <- iconv(data_col, from = "UTF-8", to = "ASCII", sub = "")  
  # Remove non-alphanumeric characters except hyphens  
  data_col <- gsub("[^a-zA-Z0-9 \\-]", "", data_col)  
  # Remove extra spaces  
  data_col <- gsub("\\s+", " ", data_col)  
  # Trim leading and trailing spaces  
  trimws(data_col)  
}  
  
# Create a new dataframe with only the cleaned Resort column  
cleaned_resort_df <- data.frame(CleanedResort = clean_non_alphanumeric(resort_df$Resort))  
  
# Add the original Resort names for comparison  
cleaned_resort_df$OriginalResort <- resort_df$Resort  
  
# Write the cleaned data to a new CSV file  
write.csv(cleaned_resort_df, "cleaned_resort_names.csv", row.names = FALSE)  
  
# Rename ID column to Resort_ID  
resort_df <- resort_df |>  
  rename(Resort_ID = ID)  
  
resort_df <- resort_df |>  
  mutate(clean_resort_name = clean_non_alphanumeric(Resort))
```

```

#Removing NAs and weird values
resort_df <- resort_df |> filter(Price > 0)

# Some ski resorts have "Unknown" for their ski season. For these we set a sensible value if

resort_df <- resort_df %>%
  mutate(Hemisphere = ifelse(Latitude > 0, "Northern", "Southern"))

# Function to process Season column
process_season <- function(season) {
  if (is.na(season)) return(NULL)

  # Handle "Year-round"
  if (tolower(season) == "year-round") {
    return(data.frame(Start = 1, End = 12))
  } else if (tolower(season) == "unknown") {
    return(data.frame(Start = NA, End = NA))
  }

  # Split multiple intervals
  intervals <- strsplit(season, "\\s*")[[1]]

  # Parse each interval
  parsed_intervals <- lapply(intervals, function(interval) {
    months <- str_extract_all(interval, "[A-Za-z]+")[[1]]
    if (length(months) == 1) {
      # Single-month season
      start <- match(months, month.name)
      return(data.frame(Start = start, End = start))
    } else if (length(months) == 2) {
      # Range of months
      start <- match(months[1], month.name)
      end <- match(months[2], month.name)
      return(data.frame(Start = start, End = end))
    }
    return(NULL)
  })

  # Combine parsed intervals
  bind_rows(parsed_intervals)
}

```

```

# Process all seasons and expand intervals
season_data <- resort_df %>%
  mutate(
    Season_Data = map(Season, process_season)
  ) %>%
  unnest(Season_Data)

# Calculate average season start and end by hemisphere
average_season_hemisphere <- season_data %>%
  group_by(Hemisphere) %>%
  summarise(
    Avg_Season_Start = mean(Start, na.rm = TRUE),
    Avg_Season_End = mean(End, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Avg_Season = paste(month.name[round(Avg_Season_Start)], "-", month.name[round(Avg_Season_End)])
  )

# Calculate average season start and end by continent
average_season_continent <- season_data %>%
  group_by(Continent) %>%
  summarise(
    Avg_Season_Start = mean(Start, na.rm = TRUE),
    Avg_Season_End = mean(End, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    Avg_Season = paste(month.name[round(Avg_Season_Start)], "-", month.name[round(Avg_Season_End)])
  )

# Print results
print("Average season by hemisphere:")

```

```
[1] "Average season by hemisphere:"
```

```
print(average_season_hemisphere)
```

```

# A tibble: 2 x 4
  Hemisphere Avg_Season_Start Avg_Season_End Avg_Season

```

	<chr>	<dbl>	<dbl> <chr>
1	Northern	11.2	4.24 November - April
2	Southern	6.06	9.69 June - October

```
print("Average season by continent:")
```

```
[1] "Average season by continent:"
```

```
print(average_season_continent)
```

```
# A tibble: 5 x 4
  Continent Avg_Season_Start Avg_Season_End Avg_Season
  <chr>          <dbl>          <dbl> <chr>
1 Asia          11.5            4.57 December - May
2 Europe         11.2            4.25 November - April
3 North America  11.2            4.16 November - April
4 Oceania         6.11            9.56 June - October
5 South America  6                9.86 June - October
```

```
# Extract the average season for each hemisphere
northern_avg_season <- average_season_hemisphere %>%
  filter(Hemisphere == "Northern") %>%
  pull(Avg_Season)

southern_avg_season <- average_season_hemisphere %>%
  filter(Hemisphere == "Southern") %>%
  pull(Avg_Season)

# Assign default seasons to "Unknown" based on hemisphere averages
resort_df <- resort_df %>%
  mutate(
    Season = case_when(
      Season == "Unknown" & Hemisphere == "Northern" ~ northern_avg_season,
      Season == "Unknown" & Hemisphere == "Southern" ~ southern_avg_season,
      Season == "Year-round" ~ "January - December",
      TRUE ~ Season # Keep other seasons as is
    )
  )
#Now resort_df should have no missing or incorrect values, and seasons are now consistent
```

```

# Function to assign resort IDs based on latitude and longitude with a specified tolerance
assign_resort_ids <- function(snow_data, resort_data, tolerance = 0.1) {
  # Initialize Resort_ID as a character column with empty strings
  snow_data <- snow_data %>%
    mutate(Resort_ID = "")

  for (i in seq_len(nrow(resort_data))) {
    # Get the latitude and longitude of the current resort
    resort_lat <- resort_data$Latitude[i]
    resort_lon <- resort_data$Longitude[i]
    resort_id <- resort_data$Resort_ID[i]

    # Filter snow data within the specified tolerance
    snow_data <- snow_data %>%
      mutate(
        Resort_ID = ifelse(
          Latitude >= (resort_lat - tolerance) & Latitude <= (resort_lat + tolerance) &
            Longitude >= (resort_lon - tolerance) & Longitude <= (resort_lon + tolerance),
          ifelse(
            Resort_ID == "",
            as.character(resort_id), # Assign the first ID
            paste0(Resort_ID, ", ", resort_id) # Append additional IDs
          ),
          Resort_ID
        )
      )
  }

  # Replace empty strings with NA for unmatched rows
  snow_data <- snow_data %>%
    mutate(Resort_ID = ifelse(Resort_ID == "", NA, Resort_ID))

  return(snow_data)
}

# Specify file paths for saving/loading
snow_df2_file <- "snow_df2.csv"
avg_snow_file <- "avg_snow_by_resort_and_month.csv"

# Check if `snow_df2` already exists
if (!file.exists(snow_df2_file)) {
  # Create `snow_df2` with Resort_ID assignment

```

```

snow_df2 <- snow_df %>%
  mutate(Resort_ID = NA)

# Assign Resort_IDs to snow_df
tolerance_value <- 0.5 # Set your desired tolerance
snow_df2 <- assign_resort_ids(snow_df2, resort_df, tolerance = tolerance_value)

# Save `snow_df2` to a CSV file
write_csv(snow_df2, snow_df2_file)
} else {
  # Load `snow_df2` from CSV
  snow_df2 <- read_csv(snow_df2_file)
}

```

Rows: 820522 Columns: 5

-- Column specification -----

Delimiter: ",",

chr (1): Resort_ID

dbl (3): Latitude, Longitude, Snow

date (1): Month

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```

#removing snow readings that aren't near a resort to reduce computation
snow_df_with_ID_no_NA <- snow_df2 |> filter(!is.na(Resort_ID))

```

```

# Ensure Resort_ID is a list of IDs split by commas

```

```

snow_df_with_ID_no_NA <- snow_df_with_ID_no_NA %>%

```

```

  mutate(
    Resort_ID = strsplit(as.character(Resort_ID), "\\s*") # Split IDs into lists
  )

```

```

# Expand snow_df_with_ID_no_NA to have one row per Resort_ID

```

```

expanded_snow_df <- snow_df_with_ID_no_NA %>%

```

```

  unnest(Resort_ID) %>%

```

```

  mutate(Resort_ID = as.numeric(Resort_ID)) # Convert Resort_ID to numeric for proper group

```

```

# expanded snow df contains all snow coverage readings within 55km of each resort over all 12

```

```

# we will average this so that we have an average snow coverage reading per month for each r

```

```

# note some resorts may not have an average reading for some months

```

```
# Create `avg_snow_by_resort_and_month` dataframe
if (!file.exists(avg_snow_file)) {

  # Group by Resort_ID and Month, and compute the average snow
  avg_snow_by_resort_and_month <- expanded_snow_df %>%
    group_by(Resort_ID, Month) %>%
    summarise(
      avg_snow = mean(Snow, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    arrange(Resort_ID, Month)

  # Save `avg_snow_by_resort_and_month` to a CSV file
  write_csv(avg_snow_by_resort_and_month, avg_snow_file)
} else {
  avg_snow_by_resort_and_month <- read_csv(avg_snow_file)
}
```

Rows: 4790 Columns: 3

-- Column specification -----

Delimiter: ","

dbl (2): Resort_ID, avg_snow

date (1): Month

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Ensure Resort_ID is numeric in avg_snow_by_resort_and_month
all_resort_ids <- unique(resort_df$Resort_ID)
```

```
# Find missing Resort_IDs
```

```
missing_resort_ids <- setdiff(all_resort_ids, unique(avg_snow_by_resort_and_month$Resort_ID))
```

```
# Print the missing IDs
```

```
print(missing_resort_ids) # This is the resort that has no snow data
```

[1] 51

```
kable(head(avg_snow_by_resort_and_month))
```

Resort_ID	Month	avg_snow
1	2022-01-01	99.70437
1	2022-02-01	99.82750
1	2022-03-01	99.87687
1	2022-04-01	88.82812
1	2022-05-01	82.74909
1	2022-06-01	37.92500

Using a radius of 0.5 degrees in latitude and longitude (equivalent to ~55km at the equator), we determined the average snow for each resort location for each month. Interestingly some resorts (especially in the europe region) are clustered very close together - within this 55km radius. As such, for the weather stations that lie in an overlapping region between 2 nearby resorts we have an average snow reading that applied to both resorts. For each resort we take all weather stations within our tolerance (± 0.5 deg latitude and longitude) and average their readings for each month.

Some resorts don't have snow coverage readings for all months. This is presumably due to outages for weather cells located in those areas.

Data Analysis Plots

Resort location on world map and snow readings

```
#plot the latitude and longitude of resorts as well as the latitude and longitude in the snow
# Extract unique latitude and longitude from the snow dataset
unique_snow_df <- snow_df %>%
  distinct(Latitude, Longitude)

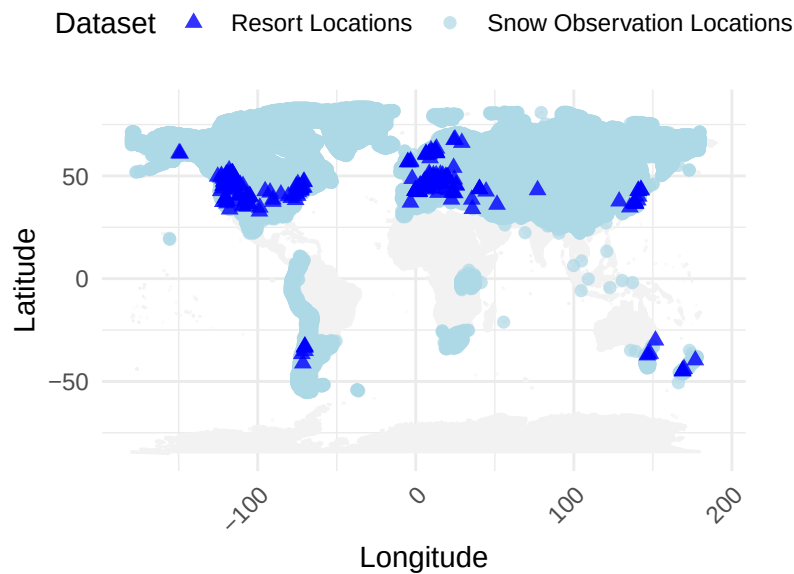
# Plot the latitude and longitude of resorts and snow observation locations over a world map
ggplot() +
  # Add the world map as a base layer
  borders("world", colour = "gray95", fill = "gray95") +
  coord_fixed(1.3) + # Set aspect ratio for better map projection
  # Plot snow observation locations
  geom_point(data = unique_snow_df, aes(x = Longitude, y = Latitude, color = "Snow Observati
    alpha = 0.7, size = 2, shape = 16) +
  # Plot resort locations
```

```

geom_point(data = resort_df, aes(x = Longitude, y = Latitude, color = "Resort Locations"),
           alpha = 0.8, size = 2, shape = 17) +
# Customize colors for points
scale_color_manual(
  values = c("Resort Locations" = "blue", "Snow Observation Locations" = "lightblue"),
  name = "Dataset"
) +
# Add labels and styling
labs(
  title = "Global Map of Resorts and Snow Observations",
  x = "Longitude",
  y = "Latitude"
) +
theme_minimal() +
theme(
  legend.position = "top",
  plot.title = element_text(size = 16, face = "bold", hjust = 0.5),
  axis.text.x = element_text(angle = 45, hjust = 1)
)

```

Global Map of Resorts and Snow Observations



It is clear that there are many more snow weather observation stations across the whole world than just at the resort locations. When merging the 2 dataframes we only need to consider the snow observations for gps locations where there is a resort, so we will assign these observations

with an ID that corresponds to the resort that is being measured. For snow observations at a longitude or latitude that don't contain a resort will have an ID value of NaN.

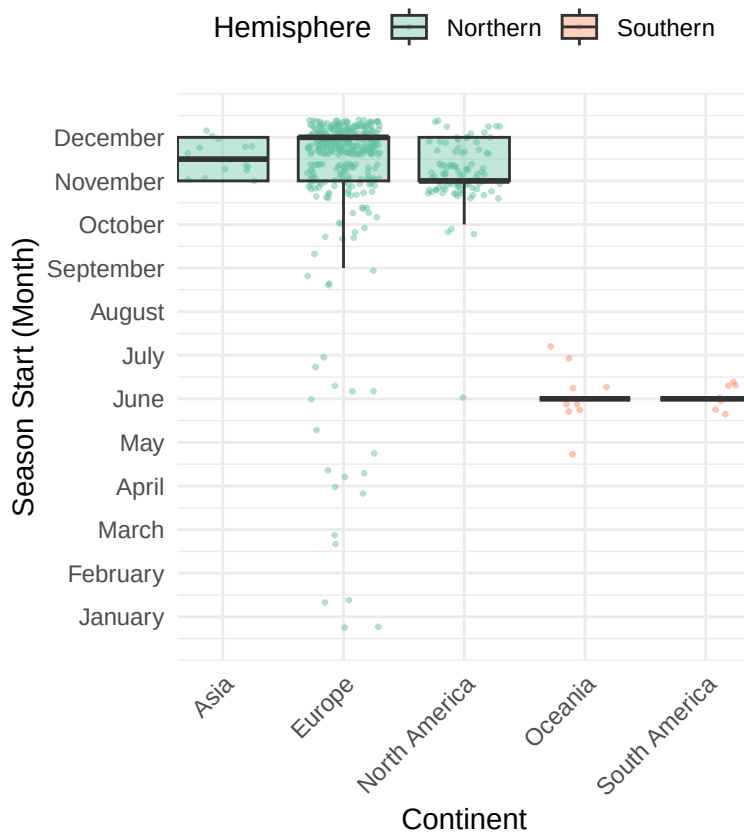
Average Ski Season Start and End by Continent

```
# Create the boxplots for start and end
ggplot(season_data, aes(x = Continent, y = Start, fill = Hemisphere)) +
  geom_jitter(aes(color = Hemisphere), width = 0.3, alpha = 0.5, size = 0.5) +
  geom_boxplot(alpha = 0.4, outlier.shape = NA, coef=2) +
  labs(
    title = "Ski Season Start by Continent",
    x = "Continent",
    y = "Season Start (Month)",
    fill = "Hemisphere",
    color = "Hemisphere",
    caption = "Boxplots show the distribution of ski season start months across continents."
  ) +
  scale_y_continuous(
    breaks = 1:12,
    labels = month.name,
    limits = c(0, 13)
  ) +
  scale_fill_brewer(palette = "Set2") +
  scale_color_brewer(palette = "Set2") +
  coord_cartesian(expand = FALSE) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1),
    plot.caption = element_text(size=8),
    legend.position = "top"
  )
)
```

Warning: Removed 25 rows containing non-finite outside the scale range
(`stat\_boxplot()`).

Warning: Removed 25 rows containing missing values or values outside the scale range
(`geom\_point()`).

Ski Season Start by Continent



Boxplots show the distribution of ski season start months across continents.

```
ggplot(season_data, aes(x = Continent, y = End, fill = Hemisphere)) +
  geom_jitter(aes(color = Hemisphere), width = 0.3, alpha = 0.5, size = 0.5) +
  geom_boxplot(alpha = 0.4, outlier.shape = NA, coef=4) +
  labs(
    title = "Ski Season End by Continent",
    x = "Continent",
    y = "Season End (Month)",
    fill = "Hemisphere",
    color = "Hemisphere",
    caption = "Boxplots show the distribution of ski season end months across continents."
  ) +
  scale_y_continuous(
    breaks = 1:12,
    labels = month.name,
    limits = c(0, 13)
```

```

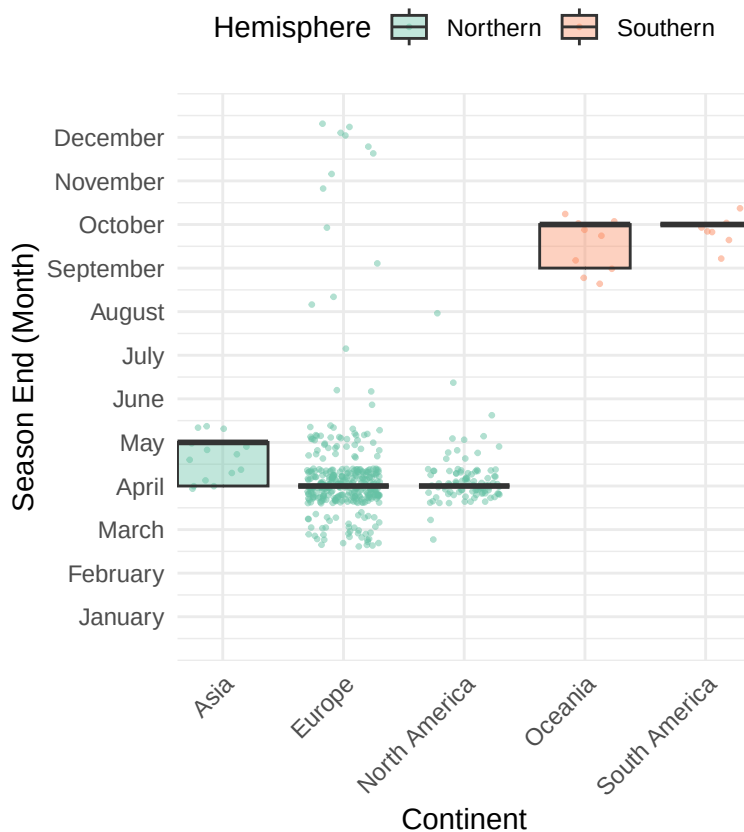
) +
scale_fill_brewer(palette = "Set2") +
scale_color_brewer(palette = "Set2") +
coord_cartesian(expand = FALSE) +
theme_minimal() +
theme(
  plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
  axis.text.x = element_text(angle = 45, hjust = 1),
  plot.caption = element_text(size=8),
  legend.position = "top",
)

```

Warning: Removed 25 rows containing non-finite outside the scale range
(`stat\_boxplot()`).

Removed 25 rows containing missing values or values outside the scale range
(`geom\_point()`).

Ski Season End by Continent



Boxplots show the distribution of ski season end months across continents.

Snow coverage by month for each continent

```
# Join avg_snow_by_resort_and_month with resort_df, selecting only Continent and ID columns
avg_snow_with_continent <- avg_snow_by_resort_and_month %>%
  left_join(resort_df %>% select(Resort_ID, Continent),
            by = c("Resort_ID" = "Resort_ID"))

# Summarize average snow coverage per month for each continent
avg_snow_by_continent <- avg_snow_with_continent %>%
  group_by(Continent, Month) %>%
  summarise(
    avg_snow_coverage = mean(avg_snow, na.rm = TRUE), # Calculate average snow coverage
    .groups = "drop"
```

```

)

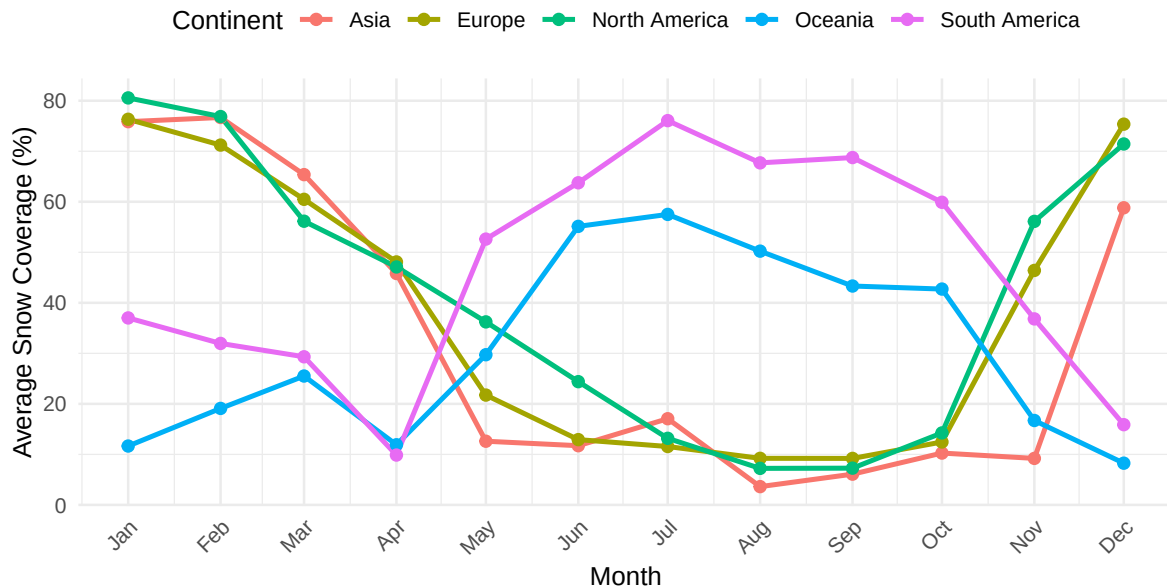
# Convert Month to a proper date format for better plotting
avg_snow_by_continent <- avg_snow_by_continent %>%
  mutate(Month = as.Date(Month, format = "%Y-%m-%d"))

# Plot the line plot showing average snow coverage per month by continent
ggplot(avg_snow_by_continent, aes(x = Month, y = avg_snow_coverage, color = Continent, group
  geom_line(size = 1) +
  geom_point(size = 2) +
  labs(
    title = "Average Snow Coverage Per Month by Continent",
    x = "Month",
    y = "Average Snow Coverage (%)",
    color = "Continent"
  ) +
  scale_x_date(date_labels = "%b", date_breaks = "1 month") + # Show month names
  theme_minimal() +
  theme(
    plot.title = element_text(size = 16, face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "top"
  )
)

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.

Average Snow Coverage Per Month by Continent



Correlation Heatmap

```
# Filter numeric columns excluding Latitude, Longitude, and Resort_ID
numeric_columns <- resort_df %>%
  select_if(is.numeric) %>%
  select(-Latitude, -Longitude, -Resort_ID)

# Calculate the correlation matrix
correlation_matrix <- cor(numeric_columns, use = "pairwise.complete.obs")

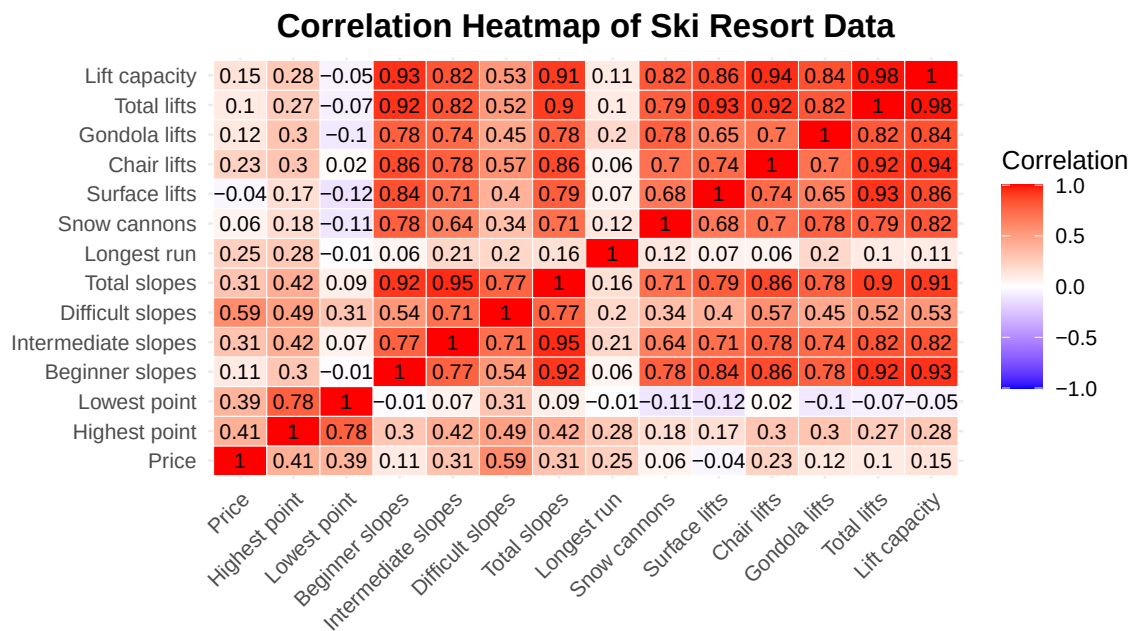
# Melt the correlation matrix for ggplot2
melted_correlation <- melt(correlation_matrix, na.rm = TRUE)

# Plot the heatmap for the lower triangular matrix
ggplot(melted_correlation, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile(color = "white") + # Heatmap tiles
  geom_text(aes(label = round(value, 2)), color = "black", size = 3) + # Add correlation va
  scale_fill_gradient2(
    low = "blue", high = "red", mid = "white",
    midpoint = 0, limit = c(-1, 1), space = "Lab",
    name = "Correlation"
```

```

) +
theme_minimal() +
labs(
  title = "Correlation Heatmap of Ski Resort Data",
  x = "",
  y = ""
) +
theme(
  axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1),
  plot.title = element_text(size = 14, face = "bold", hjust = 0.5)
)

```



Linear Regression

```

# One-hot encode the Continent column
continent_one_hot <- model.matrix(~ Continent + 0, data = resort_df) %>%
  as.data.frame()

# Rename columns for clarity
colnames(continent_one_hot) <- gsub("Continent", "", colnames(continent_one_hot))

```

```

# Add the one-hot encoded columns back to the dataframe
resort_df_with_encoded_continents <- bind_cols(resort_df, continent_one_hot)

# Split the data into training and testing sets (80% training, 20% testing)
data_split <- initial_split(resort_df_with_encoded_continents, prop = 0.8)
training_data <- training(data_split)
testing_data <- testing(data_split)

# Define a linear regression model specification
lm_spec <- linear_reg() %>%
  set_engine("lm")

# Fit the model using the training data
lm_fit <- lm_spec %>%
  fit(
    Price ~ `Highest point` + `Difficult slopes` + `Lowest point` + `Beginner slopes` + `Intermediate slopes` + `Total lifts` + `Longest run` + `Snow cannons` +
    `Europe` + `Asia` + `North America` + `Oceania`,
    data = training_data
  )

# Summarize the model
lm_summary <- summary(lm_fit$fit)

# Extract the coefficients table from the summary
coefficients_df <- as.data.frame(lm_summary$coefficients) %>%
  rownames_to_column(var = "Variable") # Add variable names as a column

# Rename columns for better readability
colnames(coefficients_df) <- c("Variable", "Estimate", "Std. Error", "t value", "P-value")

# Generate a nice table using knitr::kable
kable(
  coefficients_df,
  caption = "Linear Regression Model Coefficients",
  digits = 4 # Number of decimal places
)

```

Table 5: Linear Regression Model Coefficients

Variable	Estimate	Std. Error	t value	P-value
(Intercept)	34.6076	5.6493	6.1260	0.0000
Highest point	0.0047	0.0019	2.4108	0.0164
Difficult slopes	0.2281	0.0598	3.8120	0.0002
Lowest point	-0.0013	0.0023	-0.5511	0.5819
Beginner slopes	-0.0674	0.0383	-1.7593	0.0793
Intermediate slopes	0.0300	0.0336	0.8936	0.3721
Total lifts	0.1042	0.0777	1.3414	0.1806
Longest run	0.7958	0.1894	4.2015	0.0000
Snow cannons	0.0001	0.0030	0.0235	0.9813
Europe	-8.3441	4.9812	-1.6751	0.0947
Asia	-6.0846	5.8329	-1.0432	0.2975
North America	22.4954	5.1560	4.3630	0.0000
Oceania	23.2574	6.5972	3.5254	0.0005

```
# Predict on the training and testing sets
training_data <- training_data %>%
  mutate(
    Predicted_Price_LM = predict(lm_fit, new_data = training_data)$ .pred
  )
testing_data <- testing_data %>%
  mutate(
    Predicted_Price_LM = predict(lm_fit, new_data = testing_data)$ .pred
  )

# Calculate average price by continent using the training data
continent_avg_price <- training_data %>%
  group_by(Continent) %>%
  summarise(Avg_Price = mean(Price, na.rm = TRUE), .groups = "drop")

# Add continent average price predictions to both datasets
training_data <- training_data %>%
  left_join(continent_avg_price, by = "Continent") %>%
  mutate(
    Predicted_Price_Continent = Avg_Price
  )
testing_data <- testing_data %>%
  left_join(continent_avg_price, by = "Continent") %>%
  mutate(
    Predicted_Price_Continent = Avg_Price
  )
```

```

)

# Calculate RMSE for both models on testing data
rmse_lm <- rmse(testing_data, truth = Price, estimate = Predicted_Price_LM)
rmse_continent <- rmse(testing_data, truth = Price, estimate = Predicted_Price_Continent)

# Print the RMSE values
cat("RMSE for Linear Regression Model (Testing Data):", rmse_lm$.estimate, "\n")

```

RMSE for Linear Regression Model (Testing Data): 11.80287

```

cat("RMSE for Continent Average Model (Testing Data):", rmse_continent$.estimate, "\n")

```

RMSE for Continent Average Model (Testing Data): 14.97696

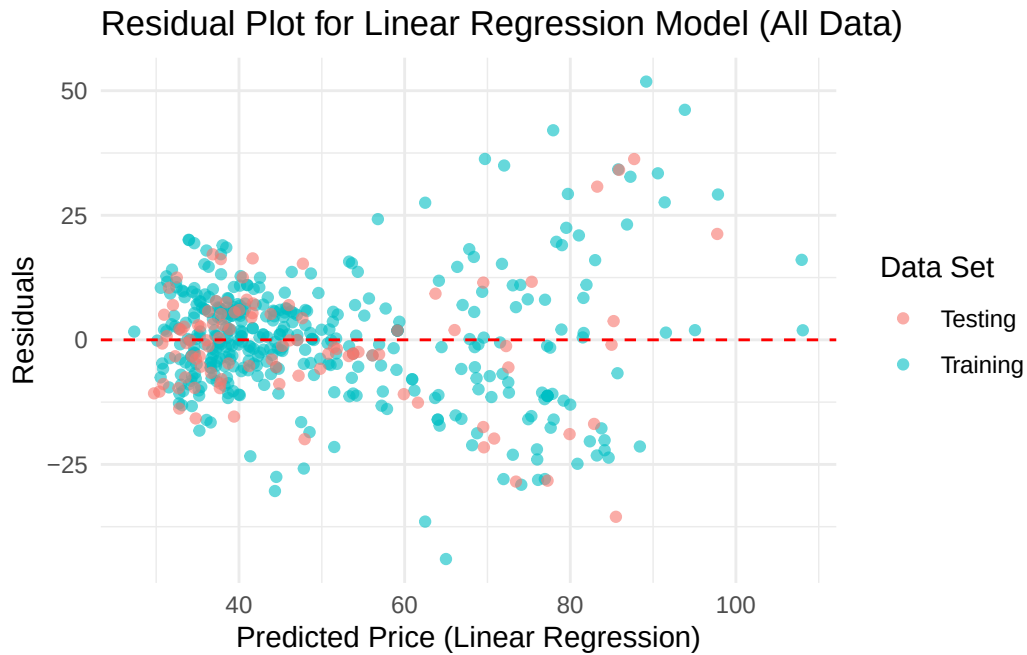
```

# Combine training and testing data for overall residual analysis
resort_df_with_predictions <- bind_rows(
  training_data %>% mutate(Data_Set = "Training"),
  testing_data %>% mutate(Data_Set = "Testing")
)

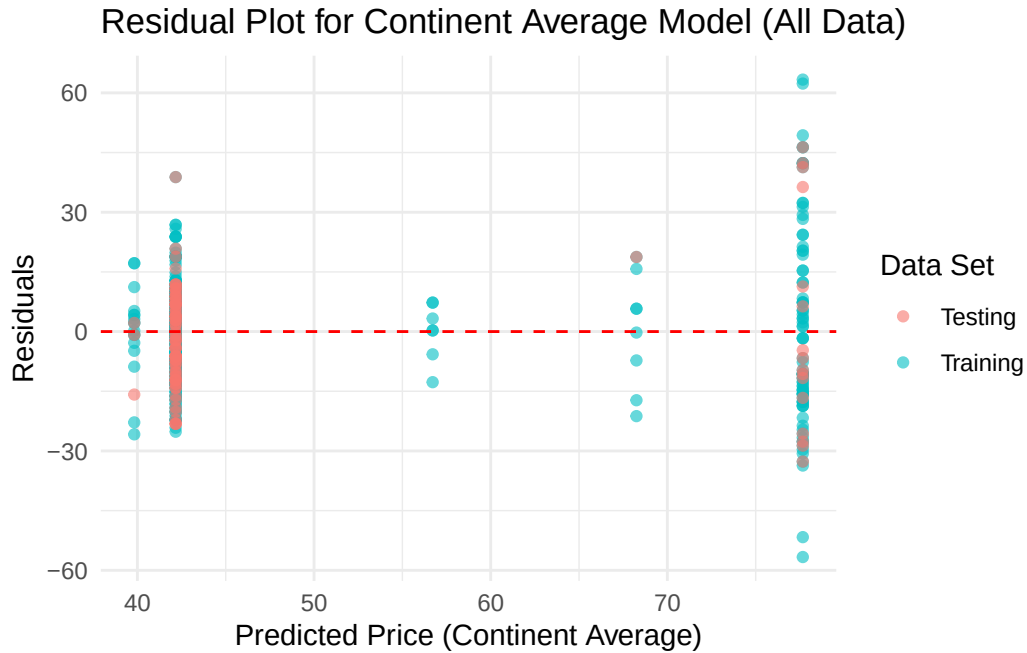
# Add residuals for both models
resort_df_with_predictions <- resort_df_with_predictions %>%
  mutate(
    Residual_LM = Price - Predicted_Price_LM,
    Residual_Continent = Price - Predicted_Price_Continent
  )

# Residual plot for Linear Regression Model
ggplot(resort_df_with_predictions, aes(x = Predicted_Price_LM, y = Residual_LM, color = Data_Set)) +
  geom_point(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(
    title = "Residual Plot for Linear Regression Model (All Data)",
    x = "Predicted Price (Linear Regression)",
    y = "Residuals",
    color = "Data Set"
  ) +
  theme_minimal()

```



```
# Residual plot for Continent Average Model
ggplot(resort_df_with_predictions, aes(x = Predicted_Price_Continent, y = Residual_Continent)) +
  geom_point(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(
    title = "Residual Plot for Continent Average Model (All Data)",
    x = "Predicted Price (Continent Average)",
    y = "Residuals",
    color = "Data Set"
  ) +
  theme_minimal()
```



```
# Calculate RMSE for both models
rmse_lm_all <- sqrt(mean(resort_df_with_predictions$Residual_LM^2, na.rm = TRUE))
rmse_continent_all <- sqrt(mean(resort_df_with_predictions$Residual_Continent^2, na.rm = TRUE))

# Display a comparison table
comparison_df <- data.frame(
  Model = c("Linear Regression", "Continent Average"),
  RMSE = c(rmse_lm_all, rmse_continent_all)
)

# Print the table with knitr
kable(
  comparison_df,
  caption = "Comparison of RMSE Between Linear Regression and Continent Average Models (All Data)"
)
```

Table 6: Comparison of RMSE Between Linear Regression and Continent Average Models (All Data)

Model	RMSE
Linear Regression	12.07909
Continent Average	14.89273

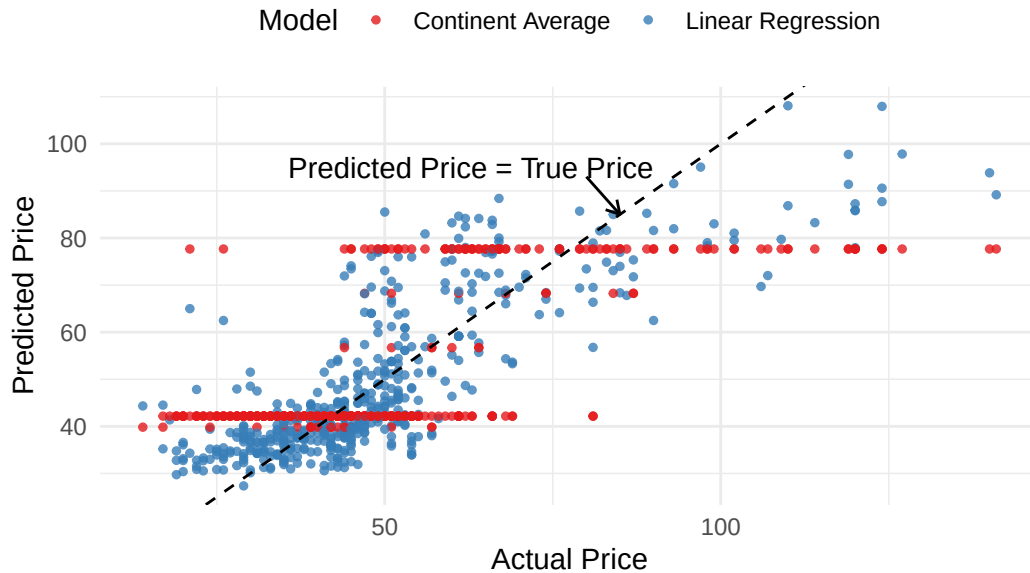
```

# Combine actual and predicted values for both models into long format
long_predictions <- resort_df_with_predictions %>%
  select(Price, Predicted_Price_LM, Predicted_Price_Continent) %>%
  pivot_longer(
    cols = c(Predicted_Price_LM, Predicted_Price_Continent),
    names_to = "Model",
    values_to = "Predicted_Price"
  ) %>%
  mutate(Model = recode(Model,
                        Predicted_Price_LM = "Linear Regression",
                        Predicted_Price_Continent = "Continent Average"))

# Scatter plot: Actual vs Predicted with annotation for y = x line
ggplot(long_predictions, aes(x = Price, y = Predicted_Price, color = Model)) +
  geom_point(alpha = 0.8, size = 1) +
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "black") +
  annotate("text", x = 90, y = 95, label = "Predicted Price = True Price", color = "black", size = 14) +
  annotate("segment", x = 80, xend = 85, y = 93, yend = 85,
           arrow = arrow(length = unit(0.2, "cm")), color = "black") +
  labs(
    title = "Actual vs Predicted Prices by Model",
    x = "Actual Price",
    y = "Predicted Price",
    color = "Model"
  ) +
  scale_color_brewer(palette = "Set1") +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
    legend.position = "top"
  )

```

Actual vs Predicted Prices by Model



```
# Combine residuals for both models into long format
long_residuals <- resort_df_with_predictions %>%
  select(Residual_LM, Residual_Continent) %>%
  pivot_longer(
    cols = c(Residual_LM, Residual_Continent),
    names_to = "Model",
    values_to = "Residual"
  ) %>%
  mutate(Model = recode(Model,
    Residual_LM = "Linear Regression",
    Residual_Continent = "Continent Average"))

# Density plot for residuals
ggplot(long_residuals, aes(x = Residual, fill = Model)) +
  geom_density(alpha = 0.6) +
  labs(
    title = "Distribution of Residuals by Model",
    x = "Residual",
    y = "Density",
    fill = "Model"
  ) +
  scale_fill_brewer(palette = "Set1") +
  theme_minimal() +
```

```
theme(  
  plot.title = element_text(size = 14, face = "bold", hjust = 0.5),  
  legend.position = "top"  
)
```

